CS 61B Fall 2024

Midterm 2

Thursday, October 24, 2024

Your Name: _____

Your SID: _____ Location: _____

SID of person to your left: _____ Right: _____

This exam is worth **100 points**, and consists of 5 questions. Questions are ordered by topic, not by difficulty.

Question:	1	2	3	4	5	Total
Points:	15	18	32	23	12	100

- ◯ indicates only one circle should be filled in. ☐ indicates more than one box may be filled in. **Please fill in the shape completely.** If you change your response, **erase as completely as possible**.
- Anything you write that you ~~cross out~~ will not be graded.
- If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade according to the worst interpretation.
- Unless otherwise specified, all data structures and algorithms behave according to their implementation in lecture, with no additional optimizations.
- If an implementation detail (e.g. tiebreaking scheme, linked list topology) is relevant, it will be explicitly noted in the question.

Coding questions:

- You may write at most one statement per blank and you may not use more blanks than provided.
- Your answer will be reformatted according to the 61B style guidelines. For example, any if-statement requires at least three lines for the purposes of determining line count.
- You may not use ternary operators, lambdas, streams, or multiple assignment.
- Unless otherwise specified, you may assume that everything on the reference card has been imported.

Write the statement below in the same handwriting you will use on the rest of the exam. Failure to do so may result in a voided exam.

> I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat. If these answers are not my own work, I may be deducted up to 9,876,543,210 points.

_____

_____

_____

_____

_____

Signature: _____

1 61Buff

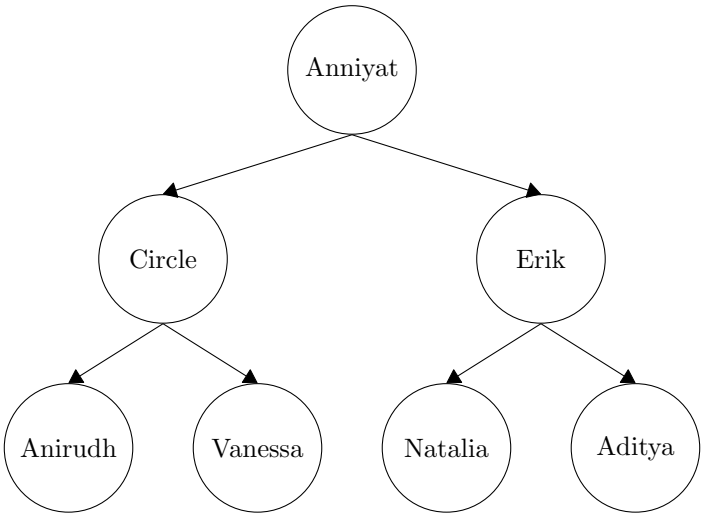
(15 points)

Aditya has organized a one-on-one arm wrestling competition amongst all of 61B's TAs. Aditya records the winner and loser of each battle and, in true 61B fashion, represents the leaderboard as a binary tree.

Each node represents a TA, and all nodes to the left of a node represent TAs who performed worse on the leaderboard, and all nodes to the right represent TAs who performed better on the leaderboard.

Below is the current leaderboard (in a list from first place to last) and the corresponding tree Aditya draws:

[Aditya, Erik, Natalia, Anniyat, Vanessa, Circle, Anirudh]



(a) (1 point) Which of the following tree traversals gives us the leaderboard in order from best arm wrestler to worst?

◯ Pre-order ◯ In-order ● None of the above

◯ Post-order ◯ Level order

Pre-order results in: [Anniyat, Circle, Anirudh, Vanessa, Erik, Natalia, Aditya]

In-order results in: [Anirudh, Circle, Vanessa, Anniyat, Erik, Natalia, Aditya]

Post-order results in: [Anirudh, Vanessa, Circle, Natalia, Aditya, Erik, Anniyat]

None of these match the leaderboard list at the top of the question.

(b) (1 point) Which of the following tree traversals gives us the reverse ordering of the leaderboard (worst to best)?

◯ Pre-order ● In-order ◯ None of the above

◯ Post-order ◯ Level order

Continuing on from the solution in the previous part, notice that the in-order traversal does result in the reverse of the leaderboard list at the top of the question.

(c) (6 points) Consider a function that finds the leader of a (potentially unbalanced) leaderboard tree. What is the asymptotic best- and worst-case runtime of this function? Answer in terms of N, the total number of TAs in the tree.

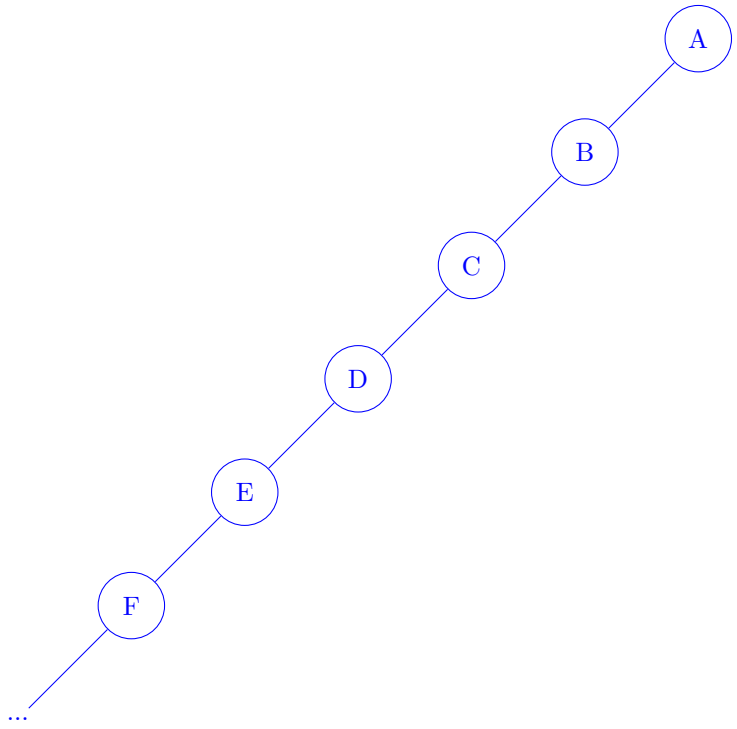
Best Case:

$\Theta(1)$

Worst Case:

$\Theta(N)$

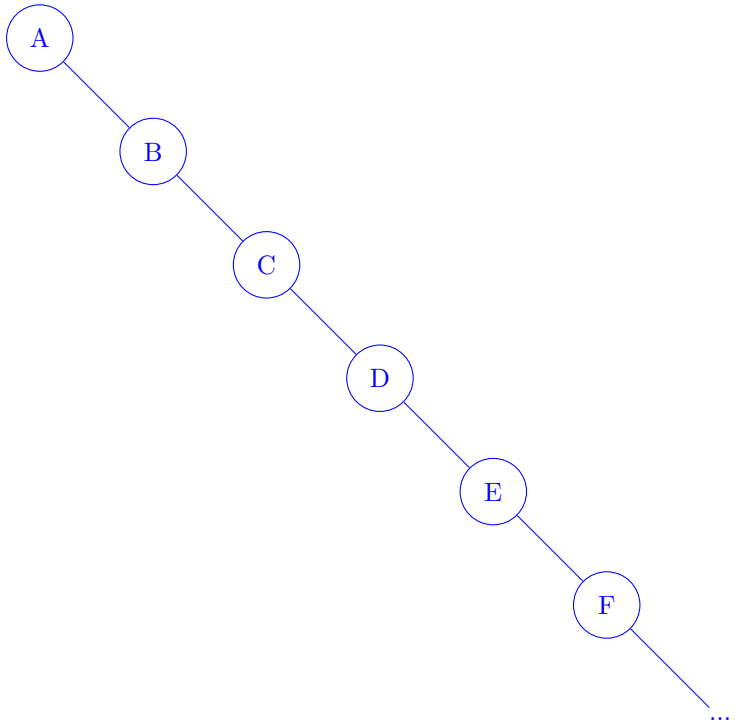
Best-case: Consider a spindly tree where every node only has a left child:



No matter big this tree grows, the leader is always A, because everyone to A's left performed worse than A.

Finding the leader in this tree takes $\Theta(1)$ time, since you just pick the root.

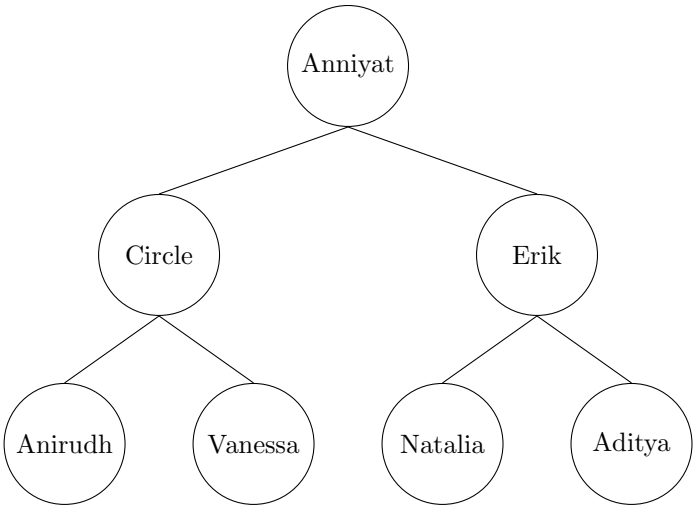
Worst-case: Consider a spindly tree where every node only has a right child:



The leader is always at the very "bottom" of this tree, because your right child did better than you did.

Finding the leader in this tree requires traversing all of its nodes to reach the bottom, which takes $\Theta(N)$ time.

The TAs don't like that Aditya has decided that he's the winner of his own competition. Thinking quickly, Aditya decides to claim that the diagram above is actually a min-heap (first place is towards the root).

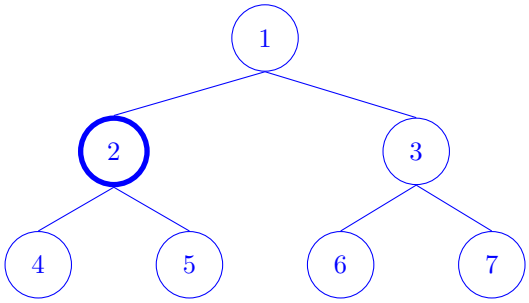


(d) (3 points) Each TA reads the heap, and decides to assume that their placement was the best possible placement that could have yielded the heap above. What is the best possible placement that each TA below could have according to the heap above?

Circle:	○ 1	● 2	○ 3	○ 4	○ 5	○ 6	○ 7
Anniyat:	● 1	○ 2	○ 3	○ 4	○ 5	○ 6	○ 7

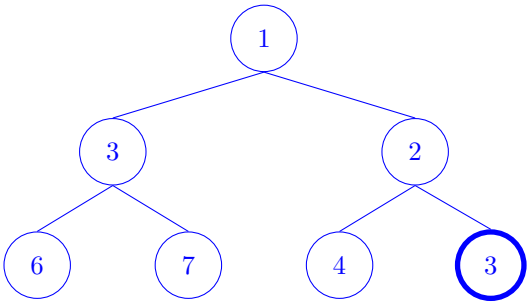
Aditya: ◯ 1 ◯ 2 ● 3 ◯ 4 ◯ 5 ◯ 6 ◯ 7

Circle cannot be 1st place, because Anniyat is above him in the heap. However, Circle could be 2nd place. The example heap below shows this (Circle's node in bold).



Anniyat must be 1st place, since he is at the top of the heap. This is the only possible placement for Anniyat.

Aditya cannot be 1st or 2nd place, since there are two people (Erik and Anniyat) above him in the heap. However, Aditya could be 3rd place. The example heap below shows this (Aditya's node in bold).



Now the TAs are complaining that the leaderboard heap is ambiguous. Aditya decides to show the match results as a directed graph instead, where:

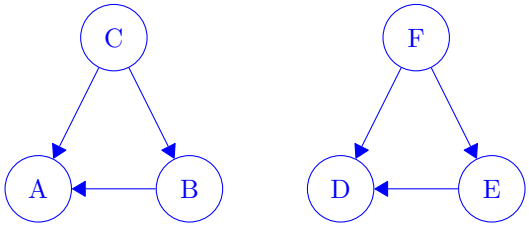
- Each vertex of the graph represents a TA.
- Each edge represents the outcome of a match, pointing from the winner to loser.
- Two TAs cannot face each other twice.

(e) (2 points) If there are no cycles in this graph, then there is always an unambiguous first-place TA.

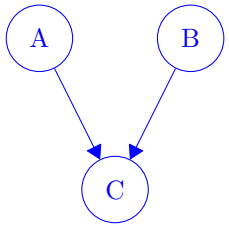
◯ True ● False

False. Here are some simple counterexamples:

Counterexample 1: Disconnected components. Even though C and F seem to be the winners of their own components, we can't tell who is first place between C and F.



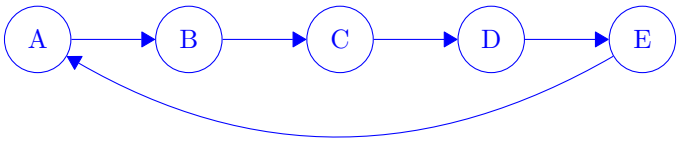
Counterexample 2: This graph has no cycles, but we can't tell which of A and B should be in first place.



(f) (2 points) If there are as many edges as there are vertices in the graph, then each TA must have faced every other TA in a match exactly once.

◯ True ● False

False. Here is a simple counterexample:



This graph has 5 vertices and 5 edges, but not all TAs have faced each other once.

For example, B and D have never faced each other. (Many other TAs also have not faced each other.)

2 Disjoint Runtimes

(18 points)

In each subpart, determine the asymptotic runtime behavior. Answer in Θ notation, with respect to N.

For all Weighted Quick Union variants, tiebreak `connect` by connecting the lower-indexed root underneath the higher-indexed root.

(a) (6 points)

```
public static void f1(int N, QuickUnion qu) {
    Random rand = new Random();
    for (int i = 0; i < N; i += 1) {
        qu.connect(rand.nextInt(N), rand.nextInt(N));
    }
    for (int i = 0; i < N; i += 1) {
        if (!qu.isConnected(rand.nextInt(N), rand.nextInt(N))) {
            System.out.println("Dylan");
        }
    }
}
```

Best Case:

$\Theta(N)$

Worst Case:

$\Theta(N^2)$

Best case:

Suppose `rand.nextInt(N)` always returns 0.

In the first for loop, `qu.connect(0, 0)` takes constant time on each iteration. Every item is in a set by itself, so 0 is the root of its own set. No tree-climbing is needed to find the root of the set with 0. The connect operation does nothing, so every item continues to be in a set by itself.

Since we run `qu.connect(0, 0)` in a loop N times, the first for loop takes $\Theta(N)$ time.

Then, we move on to the second for loop.

`qu.isConnected(0, 0)` also takes constant time. Every item is in a set by itself, so again, no tree-climbing is needed to find the root of the set with 0.

Since we run `qu.isConnected(0, 0)` in a loop N times, the second for loop takes $\Theta(N)$ time as well.

Each for loop takes $\Theta(N)$ time, and the two loops are separate, so the total time taken is $\Theta(N)+\Theta(N) = \Theta(N)$.

Worst case:

In the first for loop, we can create a very spindly tree.

`qu.connect(4, 3)`

`qu.connect(3, 2)`

`qu.connect(2, 1)`

`qu.connect(1, 0)`

Then, in the second for loop, we could call `qu.isConnected(4, 4)` each time.

Climbing up the entire spindly tree from the bottom (4) to the root (0) will take $\Theta(N)$ time, so a single call to `qu.isConnected(4, 4)` takes $\Theta(N)$ time.

Since the loop calls `isConnected` N times, and each call takes $\Theta(N)$ time, the total runtime of the second loop is $\Theta(N^2)$ time.

The total time taken is therefore $\Theta(N^2)$.

(b) (6 points)

```
public static void f2(int N, WeightedQuickUnion wqu) {
    Random rand = new Random();
    for (int i = 0; i < N; i += 1) {
        wqu.connect(rand.nextInt(N), rand.nextInt(N));
    }
    for (int i = 0; i < N; i += 1) {
        if (!wqu.isConnected(rand.nextInt(N), rand.nextInt(N))) {
            System.out.println("Gaston");
        }
    }
}
```

Best Case:

$\Theta(N)$

Worst Case:

$\Theta(N \log N)$

The best case is still $\Theta(N)$, using the exact same numbers as the first subpart. If every item stays in its own set, we still get the $\Theta(N)$ runtime.

With a WQU object, we can't create the spindly tree anymore. The worst-case tree height is now $\Theta(\log N)$, so each single call to `isConnected` takes $\Theta(\log N)$. Therefore, N calls takes $\Theta(N \log N)$ time.

(c) (6 points) As a reminder, the `isConnected` and `connect` methods of `WeightedQuickUnionPathCompression` each run in $\Theta(\alpha(N))$ amortized time.

```
public static void f3(int N, WeightedQuickUnionPathCompression wqu) {
    Random rand = new Random();
    for (int i = 0; i < N; i += 1) {
        wqu.connect(rand.nextInt(N), rand.nextInt(N));
    }
    for (int i = 0; i < N; i += 1) {
        if (!wqu.isConnected(rand.nextInt(N), rand.nextInt(N))) {
            System.out.println("Hamuy");
        }
    }
}
```

Best Case:

$\Theta(N)$

Worst Case:

$\Theta(N\alpha(N))$

The best case is still $\Theta(N)$, using the exact same numbers as the first subpart. If every item stays in its own set, we still get the $\Theta(N)$ runtime.

With path compression, the worst-case tree height is now $\Theta(\alpha(N))$, so each single call to `isConnected` takes $\Theta(\alpha(N))$. Therefore, N calls takes $\Theta(N\alpha(N))$ time.

This page is intentionally left (mostly) blank.

3 Unstable Matching Algorithm

(32 points)

In preparation for Project 3, Anniyat decides to devise a new partner-matching scheme.

In order to design our scheme, we first make a `StrangeHashSet<E>` which behaves as a `HashSet`, except for the following ways:

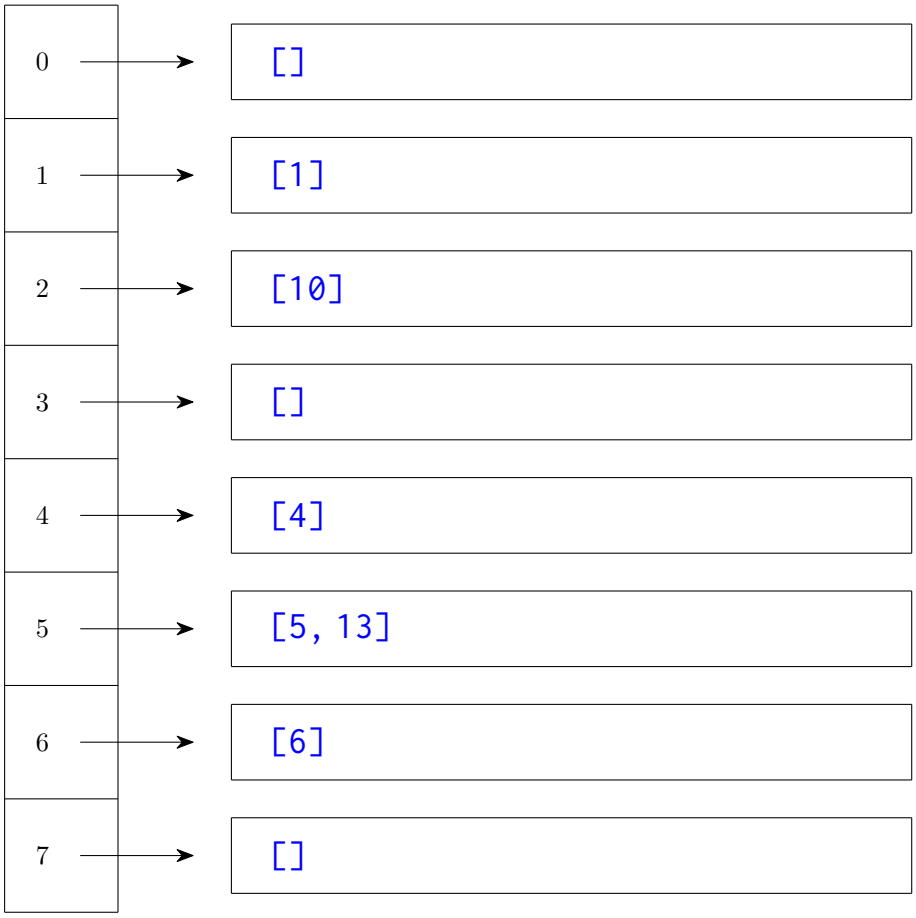
1. The `StrangeHashSet` starts with **one** bucket.
2. The `StrangeHashSet` resizes when **any bucket contains 3 or more elements**. This can trigger multiple resizes in a single `add` operation.
3. When the `StrangeHashSet` resizes, the number of buckets is **doubled**.
4. The `StrangeHashSet` has a method `public List<E>[] getBuckets()` which returns an array of Lists. The ith List contains the values stored in the ith bucket of the `StrangeHashSet` (in any order).

(a) (3 points) We create a new `StrangeHashSet<Integer>` and insert the following elements in the given order:

```
[5, 6, 1, 10, 13, 4]
```

What is the result of calling `getBuckets()`? In each box, write a list like `[5, 4]`, or write `[]` if the bucket is empty.

Note: The `hashCode` of an Integer is itself.



To solve this question, we just need to follow the StrangeHashSet behavior described in the question.

We start with one bucket:

- Bucket 0: `[]`

We add 5:

- Bucket 0: `[5]`

Then, we add 6:

- Bucket 0: `[5, 6]`

Then, we add 1:

- Bucket 0: `[5, 6, 1]`

At this point, a bucket contains 3 or more elements, so we need to resize by doubling the number of buckets:

- Bucket 0: `[6]`
- Bucket 1: `[5, 1]`

Then, we can add 10:

- Bucket 0: `[6, 10]`
- Bucket 1: `[5, 1]`

Then, we add 13:

- Bucket 0: `[6, 10]`
- Bucket 1: `[5, 1, 13]`

A bucket contains 3 or more elements, so we need to resize:

- Bucket 0: `[]`
- Bucket 1: `[5, 1, 13]`
- Bucket 2: `[6, 10]`
- Bucket 3: `[]`

A bucket still contains 3 or more elements, so we need to resize again:

- Bucket 0: `[]`
- Bucket 1: `[1]`
- Bucket 2: `[10]`
- Bucket 3: `[]`
- Bucket 4: `[]`
- Bucket 5: `[5, 13]`
- Bucket 6: `[6]`
- Bucket 7: `[]`

Then, we add 4:

- Bucket 0: `[]`

- Bucket 1: `[1]`
- Bucket 2: `[10]`
- Bucket 3: `[]`
- Bucket 4: `[4]`
- Bucket 5: `[5, 13]`
- Bucket 6: `[6]`
- Bucket 7: `[]`

We've added all the items, and no bucket contains 3 or more items, so we are done.

To set up our matching algorithm, each student is asked to submit a **unique** password. These are then stored in a (regular) `Set<String> passwords`.

We then run the following matchmaking algorithm:

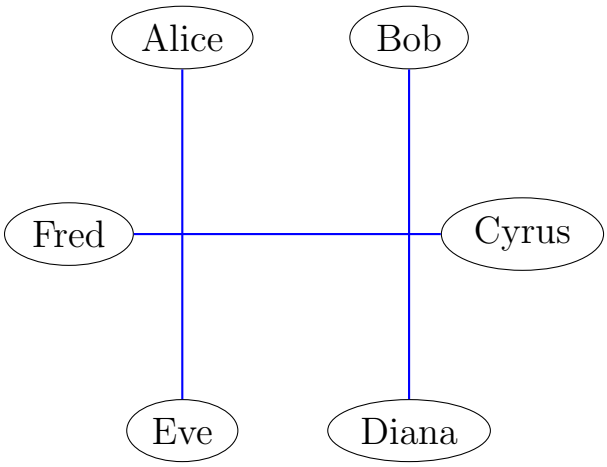
```
public class Matching {
    /**
     * Pairs the student with password a, and the student with password b.
     * If a is paired with b, then b is also paired with a.
     */
    private static void pair(String a, String b) { ... } // Implementation omitted.

    /**
     * Pairs all students according to their passwords.
     * If there are an odd number of students, pairs all but one student,
     * and returns the password of the unpaired student.
     * If there are an even number of students, pairs all students, and returns null.
     */
    public static String pairAllStudents(Set<String> passwords) {
        while (passwords.size() > 1) {
            // 1: Create a new StrangeHashSet.
            StrangeHashSet<String> matcher = new StrangeHashSet<>();

            // 2: Insert all passwords into the StrangeHashSet.
            for (String s : passwords) {
                matcher.add(s);
            }

            // 3: If any bucket contains exactly two passwords, pair the corresponding students.
            //    Then remove both passwords from passwords.
            for (List<String> bkt : matcher.getBuckets()) {
                if (bkt.size() == 2) {
                    pair(bkt.get(0), bkt.get(1));
                    passwords.remove(bkt.get(0));
                    passwords.remove(bkt.get(1));
                }
            }
```

```
        } // 4: Repeat Steps 1-3 until there are fewer than two students unpaired.

        // 5: If there's one student unpaired, return the password of that student.
        if (passwords.size() == 1) {
            return passwords.toArray()[0];
        }

        // 6: Otherwise return null.
        return null;
    }
}
```

(b) (3 points) We run `pairAllStudents` on the following students:

- Alice submits a password whose hash is 5.
- Bob submits a password whose hash is 6.
- Cyrus submits a password whose hash is 1.
- Diana submits a password whose hash is 10.
- Eve submits a password whose hash is 13.
- Fred submits a password whose hash is 4.

What are the resulting pairs? Draw 3 lines to match the students into pairs.



To solve this question, we just need to follow the `pairAllStudents` code described in the question.

Steps 1-2: We create a new StrangeHashSet and insert all passwords into the StrangeHashSet. We actually already did this in the previous subpart:

- Bucket 0: `[]`
- Bucket 1: `[1]`
- Bucket 2: `[10]`
- Bucket 3: `[]`
- Bucket 4: `[4]`
- Bucket 5: `[5, 13]`
- Bucket 6: `[6]`
- Bucket 7: `[]`

Step 3: If any bucket contains exactly two passwords, pair the corresponding students. Then remove both passwords.

Bucket 5 has exactly two passwords, so `[5, 13]` can be paired. This means that Alice (5) and Eve (13) are paired. We can also remove 5 and 13 from `passwords`.

Step 4: Repeat Steps 1-3.

Steps 1-2: We create a new StrangeHashSet and insert all passwords into the StrangeHashSet. Remember that 5 and 13 are removed, so we should only add `[6, 1, 10, 4]`.

Adding 6, 1, 10:

- Bucket 0: `[6, 1, 10]`

This triggers a resize:

- Bucket 0: `[6, 10]`
- Bucket 1: `[1]`

Adding 4:

- Bucket 0: `[6, 10, 4]`
- Bucket 1: `[1]`

This triggers another resize:

- Bucket 0: `[4]`
- Bucket 1: `[1]`
- Bucket 2: `[6, 10]`
- Bucket 3: `[]`

No bucket has 3 or more items, and we added all the items, so we're done with this step.

Step 3: If any bucket contains exactly two passwords, pair the corresponding students. Then remove both passwords.

Bucket 2 has exactly two passwords, so `[6, 10]` can be paired. This means that Bob (6) and Diana (10) are paired. We can also remove 6 and 10 from `passwords`.

Step 4: Repeat Steps 1-3.

Steps 1-2: We create a new StrangeHashSet and insert all passwords into the StrangeHashSet. Remember that 6 and 10 are removed, so we should only add `[1, 4]`.

Adding 1, 4:

- Bucket 0: `[1, 4]`

No bucket has 3 or more items, and we added all the items, so we're done with this step.

Step 3: If any bucket contains exactly two passwords, pair the corresponding students. Then remove both passwords.

Bucket 0 has exactly two passwords, so `[1, 4]` can be paired. This means that Cyrus (1) and Fred (4) are paired. We can also remove 1 and 4 from `passwords`.

Steps 5-6: At this point, everyone has been matched, so `while (passwords.size() > 1)` is false (there are no more passwords left to match). We leave the while loop, and we are done.

For the rest of the question, assume that all passwords have distinct hash codes, unless otherwise stated.

(c) (12 points) Which of the following statements are true? Select all that apply.

■ Assume that hash codes range from 0 to $2^b - 1$ for some integer b. When inserting all students into a single `StrangeHashSet`, at most b resizes occur.

☐ If a bucket contains three items immediately before a resize, it is possible for the items to end up in three different buckets after the resize.

■ In each iteration of the `while` loop, at least one pair of students will be created.

□ In each iteration of the `while` loop, at least half of the remaining students will be paired.

■ If an iteration of the `while` loop requires k resizes when adding all students to the `StrangeHashSet`, then the next iteration of the `while` loop will require strictly fewer than k resizes.

■ `pairAllStudents` is guaranteed to terminate.

◯ None of the above

Option 1: True.

"Assume that hash codes range from 0 to $2^b - 1$ for some integer b. When inserting all students into a single `StrangeHashSet`, at most b resizes occur."

If the StrangeHashSet resizes 1 time, this results in 2 buckets.

If the StrangeHashSet resizes 2 times, this results in $2^2 = 4$ buckets.

If the StrangeHashSet resizes 3 times, this results in $2^3 = 8$ buckets.

If the StrangeHashSet resizes b times, then this results in 2^b buckets.

Because all passwords have distinct hash codes, in the worst case, there will be 2^b students (with hash codes from 0 to $2^b - 1$).

After b resizes, there are 2^b buckets. Since there are 2^b students with distinct hash codes, each student ends up in their own bucket.

At this point, no bucket contains 3 or more items, so no resizes will occur. Therefore, no more than b resizes will occur, and the statement is true.

Option 2: False.

"If a bucket contains three items immediately before a resize, it is possible for the items to end up in three different buckets after the resize."

All items in one bucket will get distributed between two buckets after resizing. Therefore, it's impossible for the items to end up in three different buckets.

Examples:

To see why, consider this example resize:

- Bucket 0: `[0, 2, 4]`
- Bucket 1: `[]`

After resizing:

- Bucket 0: `[0, 4]`
- Bucket 1: `[]`
- Bucket 2: `[2]`
- Bucket 3: `[]`

Notice that all of Bucket 0's items either stayed in Bucket 0, or moved to Bucket 2.

Or, to try another example:

- Bucket 0: `[]`
- Bucket 1: `[5, 9, 13]`
- Bucket 2: `[]`
- Bucket 3: `[]`

After resizing:

- Bucket 0: `[]`
- Bucket 1: `[9]`
- Bucket 2: `[]`
- Bucket 3: `[]`
- Bucket 4: `[]`
- Bucket 5: `[5, 13]`
- Bucket 6: `[]`
- Bucket 7: `[]`

Again, notice that all of Bucket 1's items either stayed in Bucket 1, or moved to Bucket 5.

If you try a few more examples, you'll notice that when we double the number of buckets, the items in any given bucket will either stay in that bucket, or move to a single other bucket.

More general argument:

To generalize, let's say that the StrangeHashTable has 8 buckets in total, and Bucket 5 has three items.

The hash codes that could go in Bucket 5 are: 5, 13, 21, 29, 37, 45, and so on.

Now, let's resize the StrangeHashTable to 16 buckets.

In the resized StrangeHashTable, the hash codes that would go in Bucket 5 are: 5, 21, 37, and so on. The hash codes that would go in Bucket 13 are: 13, 29, 45, and so on.

If you continue the pattern, you'll notice that everything that used to be in Bucket 5 must end up in either Bucket 5 or Bucket 13 after resizing.

We've just shown that all the items in one bucket will end up getting spread between two buckets after resizing. Therefore, it's impossible for the items to end up in three different buckets after resizing.

A more formal proof:

You don't need to formally prove this on an exam, but we can show this more formally by replacing our numbers with variables:

Let's say that the StrangeHashTable has n buckets in total, and bucket k has three items.

The hash codes that could go in Bucket k are $n + k$, $2n + k$, $3n + k$, $4n + k$, $5n + k$, ...

If we resize the StrangeHashTable, we now have $2n$ buckets. The hash codes $n+k$, $3n+k$, $5n+k$, $7n+k$, ... will all end up in the same bucket, since they are equivalent $\bmod 2n$. After dividing each of these hash codes by $2n$, the remainder is always $n+k$.

Similarly, the hash codes $2n+k$, $4n+k$, $6n+k$, $8n+k$, ... will all end up the same bucket, since they are equivalent $\bmod 2n$. After dividing each of these hash codes, the remainder is always k.

All the items in bucket k get redistributed to bucket k and bucket $n+k$ after resizing. Therefore, it's impossible for the items in bucket k to end up in three different buckets after resizing.

Option 3: True.

"In each iteration of the `while` loop, at least one pair of students will be created."

The while loop condition is `while (passwords.size() > 1)`, so the loop only runs if there are 2 or more students left to pair up.

If there are exactly 2 students left to pair up, they'll both be placed in Bucket 0 (remember, we start with 1 bucket), and a pair will be created.

What if there are 3 or more students? Then some resizing will need to take place as we add the students into the StrangeHashSet.

From Option 2, we know that every time a resize occurs, the 3 items in a bucket will get redistributed between two other buckets. There are two ways for the resizing to happen:

1. The items are redistributed such that 2 items end up in one bucket, and 1 item ends up in the other. After resizing, one bucket has 2 items, and the other has 1 item, so no more resizing occurs.

2. All 3 items get redistributed into the same bucket. After resizing, one bucket has 3 items, and the other has 0. This will trigger more resizes until we reach a Case 1 resize (2 items in one bucket, 1 item in the other).

From Option 1, we know that the number of resizes will be finite, so even if multiple resizes are needed (e.g. we keep hitting Case 2), we will eventually reach a Case 1 resize, and the resizing will stop.

When we reach the last Case 1 resize and stop, one of the buckets will have 2 items. Those 2 items will get paired up! This tells us that at least one pair gets created on each iteration of the while loop.

Option 4: False.

"In each iteration of the `while` loop, at least half of the remaining students will be paired."

Part (b) is a counterexample. On the first iteration, only Alice and Eve were paired. There are 6 students, and only 2 were paired, which is less than half the remaining students.

Option 5: True.

"If an iteration of the `while` loop requires k resizes when adding all students to the `StrangeHashSet`, then the next iteration of the `while` loop will require strictly fewer than k resizes."

Consider the iteration that requires k resizes. In this iteration, the kth and final resize must have taken a bucket with 3 items, and distributed those items into two buckets. One of those buckets got 2 of the

items, and the other bucket got the other 1 item, so no further resizing takes place. (We know that the 3 items won't end up in the same bucket, because that would require a $k+1$th resize, but we just said that the kth resize is the final one.)

After the kth and final resize, the bucket with 2 of the items gets paired up. Those 2 items are then removed from the StrangeHashSet before we start the next iteration.

On the next iteration, the bucket that triggered the kth resize no longer has 3 items (since we just removed 2 of them). Therefore, the kth resize will not occur, and the next iteration requires strictly fewer than k resizes.

Option 6: True.

"`pairAllStudents` is guaranteed to terminate."

This directly follows from Options 1, 3, and 5.

From Option 1, we know that the number of resizes is bounded by b (i.e. the StrangeHashTable won't get stuck resizing forever). Therefore, on the first iteration, the StrangeHashTable will resize some finite number of times.

From Option 3, we know that each time we run the loop, at least one pair of students will be created. Also, from Option 5, we know that each subsequent iteration will require strictly fewer resizes. Therefore, on each iteration, we get closer and closer to the solution (fewer unpaired students, fewer resizes needed), and after some finite number of iterations, we should reach the point where all students are paired.

(d) (2 points) Mia and Sebastian want to form a partnership, and decide to coordinate their passwords so they end up partnered (with high probability).

First, Mia selects a password whose hash is 0. What should the hash of Sebastian's password be in order to maximize the chances that he is paired with Mia? Assume that:

- Hash codes range from 0 to $2^{32}-1$.
- Students may not pick passwords with the same hash code.

2^{31}

To get Mia and Sebastian paired up, we want them to end up in the same bucket.

What if Sebastian picks hash code 2? If there are two buckets, then they end up in the same bucket:

- Bucket 0: `[0, 2]`
- Bucket 1: `[]`

But if more students exist, and the StrangeHashTable resizes to 4 buckets, then they end up in separate buckets:

- Bucket 0: `[0]`
- Bucket 1: `[]`
- Bucket 2: `[2]`
- Bucket 3: `[]`

If the table had 4 buckets, Sebastian should have picked hash code 4 (or any other multiple of 4) so that Bucket 0 has `[0, 4]`.

But what if even more students existed, and the StrangeHashTable resized to 8 buckets? Then Sebastian should pick hash code 8 (or any other multiple of 8), so that Bucket 0 has `[0, 8]`.

If we follow this logic to its conclusion, we'll see that Sebastian's hash code should be a multiple of 2, 4, 8, 16, 32, 64, and so on, so that no matter how many times the StrangeHashTable resizes, he always ends up in Bucket 0 with Mia.

Hash codes range from 0 to $2^{32} - 1$. The highest power of 2 that Sebastian can choose is 2^{31}.

With a hash code of 2^{31}, no matter how many buckets the StrangeHashTable has (could be 2, 4, 8, 16, 32, 64, etc.), Sebastian will always end up in Bucket 0 with Mia.

(e) (12 points) Consider the following modifications to `pairAllStudents`. For each modification, find a list of at most five password hashes that would cause `pairAllStudents` to infinite-loop.

Each modification is independent. If no list of password hashes can cause an infinite loop, write "Impossible".

As an example, you can write `[0, 2]` to represent a list containing one password with hash 0, and one password with hash 2.

1. Instead of **doubling** the number of buckets on a resize, we **triple** the number of buckets.

   `[0, 1, 2]`

   Most common solution: `[0, 1, 2]` and variants

   Recall that a pairing can only occur if a bucket contains exactly two items.

   If, after tripling, we can get each of the three buckets to contain exactly one item, then no pairing will be created. The algorithm will then run the while loop over and over again, always resulting in the same table (3 buckets, each with one item), and we will get stuck in an infinite loop.

   Our goal is to get one item per bucket (after tripling). In other words, in a hash table with 3 buckets, we want one item in each of the buckets.

   The sample answer achieves this. Initially, we start with one bucket:

   - Bucket 0: `[0, 1, 2]`

   The bucket now has 3 or more items, so we resize by tripling:

   - Bucket 0: `[0]`
   - Bucket 1: `[1]`
   - Bucket 2: `[2]`

   At this point, no more resizing occurs, and no pairings are created. The while loop runs again (create a new StrangeHashSet and insert the same three items). Each time we run the while loop, no pairings are created, and the loop continues forever.

   Other answers that work for the same reason (one item per bucket after tripling) include:

   - `[1, 2, 3]`

- `[1, 3, 5]`
- `[0, 4, 8]`
- `[4, 8, 9]`

More generally, we need a list of 3 hashes, where one of them is 0 mod 3, one is 1 mod 3, and one is 2 mod 3.

(Recall that if a number is 2 mod 3, this means that after dividing the number by 3, the remainder is 2. In other words, this number ends up in bucket 2. Examples of number that are 2 mod 3 include 2, 5, 8, 11, etc.)

Alternate solution 1: `[0, 3, 6]` and variants

`[0, 3, 6]` also works, after two resizes.

Initially, we start with one bucket:

- Bucket 0: `[0, 3, 6]`

The bucket now has 3 or more items, so we resize by tripling:

- Bucket 0: `[0, 3, 6]`
- Bucket 1: `[]`
- Bucket 2: `[]`

Note that all 3 items still end up in Bucket 0, because 0, 3, 6 are all 0 mod 3.

At this point, one of the buckets has 3 or more items, so we resize by tripling again:

- Bucket 0: `[0]`
- Bucket 1: `[]`
- Bucket 2: `[]`
- Bucket 3: `[3]`
- Bucket 4: `[]`
- Bucket 5: `[]`
- Bucket 6: `[6]`
- Bucket 7: `[]`
- Bucket 8: `[]`

At this point, no more resizing occurs. Also, no bucket contains exactly two items, so no pairings are created.

We move on to the next iteration of the while loop with the same three items, `[0, 3, 6]`. The same StrangeHashTable gets created (with 9 buckets), and again, no pairing is created. Each time we run the while loop, no pairings are created, and the loop continues forever.

More generally, we need a list of 3 numbers that are all 0 mod 3^n for some positive integer n. When you divide those 3 numbers by 3^n, the resulting numbers are 0 mod 3, 1 mod 3, and 2 mod 3.

Another set of alternate answers that work (but require multiple resizes per while-loop iteration) are: Any 3 numbers that are identical mod3^n, but are all different mod3^{n+1}.

Incorrect answer 1: `[1, 2, 3, 4, 5]` and variants

Five consecutive, distinct numbers like `[1, 2, 3, 4, 5]` is incorrect.

We start with one bucket:

- Bucket 0: `[1, 2, 3, 4, 5]`

After resizing by tripling:

- Bucket 0: `[3]`
- Bucket 1: `[1, 4]`
- Bucket 2: `[2, 5]`

The buckets with exactly 2 items generate pairs. `[1, 4]` is a pair, and `[2, 5]` is a pair.

This leaves just one unpaired item, `[3]`. The loop condition is `while (passwords.size() > 1)`, so at this point we'll exit the while loop and return the unpaired item. The algorithm terminates, and there is no infinite loop.

Incorrect answer 2: `[0, 1]` and variants

Any list of 2 distinct numbers is incorrect.

We start with one bucket:

- Bucket 0: `[0, 1]`

At this point, the two items get paired, and the algorithm completes.

Incorrect answer 3: Didn't follow question directions

Before part (c): "For the rest of the question, assume that all passwords have distinct hash codes, unless otherwise stated." Therefore, answers with duplicate hash codes like `[1, 1, 1]` are not correct.

In part (e): "Find a list of at most five password hashes." Any list of 6 or more numbers is not correct.

2. Two passwords are allowed to share a hash code, but no three passwords share the same hash code.

Impossible

Intuitively, if two passwords have the same hash code, then after sufficiently many resizes, they will eventually end up in their own bucket and get paired.

On an exam, one way to reach this answer is to try a few different answers (where a hash appears twice), and notice that none of them cause an infinite loop. Here are some example answers you could try:

`[0, 0]` won't work. You'll create one bucket with two items. The two items will pair, and the algorithm will terminate.

`[0, 0, 2]` won't work. You'll start with one bucket:

- Bucket 0: `[0, 0, 2]`

Then, you'll resize:

- Bucket 0: `[0, 0, 2]`
- Bucket 1: `[]`

Then you'll resize again:

- Bucket 0: `[0, 0]`
- Bucket 1: `[]`
- Bucket 2: `[2]`
- Bucket 3: `[]`

At this point, the two items in Bucket 0 will pair, and the algorithm will return the unpaired item (2) and terminate.

3. Exactly one hash code is shared by three passwords, and all other passwords have unique hash codes.

`[0, 0, 0]`

Most common solution: `[0, 0, 0]` and variants

Since the question allows us to have three duplicate hash codes, we can just try that and see what happens:

- Bucket 0: `[0, 0, 0]`

One bucket has 3 or more items, so we resize:

- Bucket 0: `[0, 0, 0]`
- Bucket 1: `[]`

Bucket 0 still has 3 or more items, so we resize:

- Bucket 0: `[0, 0, 0]`
- Bucket 1: `[]`
- Bucket 2: `[]`
- Bucket 3: `[]`

No matter how many times we resize, the 3 items will always be stuck in Bucket 0! This is an infinite loop, so we're done.

Other lists with 3 duplicate hashes also work, like `[1, 1, 1]` or `[12, 12, 12]`.

Adding other distinct numbers also works, like `[0, 0, 0, 1]` or `[0, 0, 0, 1, 2]`. The extra numbers don't change the fact that the duplicate hashes still get stuck in the same bucket forever.

Note: If you're confused about this answer, it might help to note that this infinite loop is actually different from the loops in the first modification (triple number of buckets) and the last modification (start with two buckets).

In the first and last modification, the infinite loop occurs because the while loop runs forever. We create a StrangeHashTable, and no pairings get created (because no bucket has exactly two items).

Then, we repeat Steps 1-3, creating another StrangeHashTable with the same items, and again, no pairings get created. This continues forever.

By contrast, in this modification, the infinite loop occurs because the StrangeHashSet is resizing forever. The first StrangeHashSet never finishes being created, and we never even reach the second iteration of the while loop.

4. The `StrangeHashSet` begins with **two** buckets, instead of **one** bucket.

`[0, 1]`

Most common answer: `[0, 1]` and variants

Recall that a pairing can only occur if a bucket has exactly two items.

What happens if we put one item in each bucket? No resizes occur (we only resize if a bucket contains 3 or more items). Also, no pairs get created.

The while loop then runs again, resulting in the same table (one item in each bucket) and no pairings created. The while loop runs again and again, resulting in an infinite loop.

Our goal is to get one item in each bucket. `[0, 1]` achieves this:

- Bucket 0: `[0]`
- Bucket 1: `[1]`

More generally, any answer with one even number and one odd number works. The even number goes in Bucket 0, and the odd number goes in Bucket 1. No pairs are created, and the while loop continues forever.

Alternate solution 1: `[0, 2, 4, 1]` and variants

We could also put three items in one bucket, and one item in the other bucket. As before, no bucket contains exactly two items, so no pairs are created.

The sample answer achieves this:

- Bucket 0: `[0, 2, 4]`
- Bucket 1: `[1]`

After a resize:

- Bucket 0: `[0, 4]`
- Bucket 1: `[1]`
- Bucket 2: `[2]`
- Bucket 3: `[]`

`[0, 4]` get paired, and you're left with one even number and one odd number. On the next iteration of the while loop, you'll build a StrangeHashSet with `[1, 2]`, where each bucket contains one item, and no pairs are created. The while loop will continue forever trying to unsuccessfully pair up `[1, 2]`.

More generally, any answer with 3 even numbers and 1 odd number works.

Alternatively, any answer with 3 odd numbers and 1 even number works.

Incorrect answer 1: `[0]` and variants

Note that a single item is not correct for this modification (and any other modification).

The while loop condition is `while (passwords.size() > 1)`. If the input has a single item, we skip the while loop entirely, and just return the single, unpaired item. The algorithm terminates with no infinite loop.

Incorrect answer 2: Didn't follow question directions

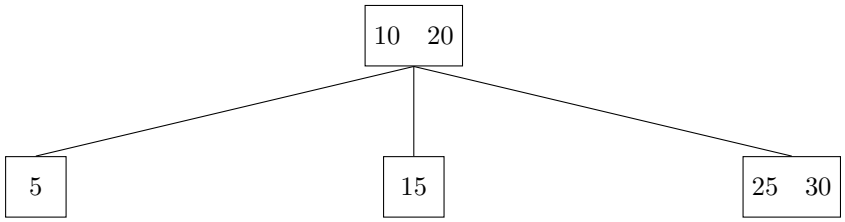
Before part (c): "For the rest of the question, assume that all passwords have distinct hash codes, unless otherwise stated." Therefore, answers with duplicate hash codes like `[1, 1, 1]` are not correct.

In part (e): "Find a list of at most five password hashes." Any list of 6 or more numbers is not correct.

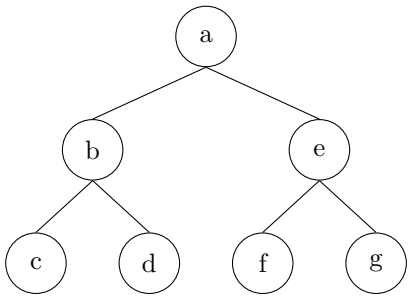
4 Fix LLRB

(23 points)

(a) (9 points) Convert the 2-3 tree below into its corresponding Left-Leaning Red-Black Tree (LLRB).



Select the value that goes in each node of the LLRB below. Not all nodes need to be used; select "None" if the node is unused.

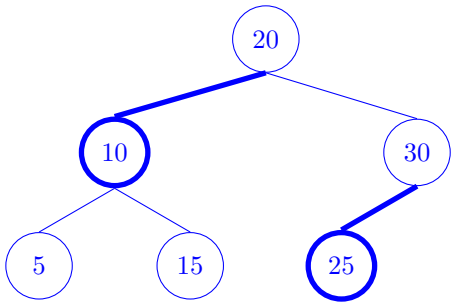


a:	○ 5	○ 10	○ 15	● 20	○ 25	○ 30	○ None
b:	○ 5	● 10	○ 15	○ 20	○ 25	○ 30	○ None
c:	● 5	○ 10	○ 15	○ 20	○ 25	○ 30	○ None
d:	○ 5	○ 10	● 15	○ 20	○ 25	○ 30	○ None
e:	○ 5	○ 10	○ 15	○ 20	○ 25	● 30	○ None
f:	○ 5	○ 10	○ 15	○ 20	● 25	○ 30	○ None
g:	○ 5	○ 10	○ 15	○ 20	○ 25	○ 30	● None

Select all red nodes:

☐ a ■ b ☐ c ☐ d ☐ e ■ f ☐ g

The tree looks like this (red nodes/edges are bolded):



(b) (14 points) Write `convert`, a function that takes in a 2-3 tree and converts it to its corresponding LLRB.

A 2-3 tree node is defined as follows:

```
public class TwoThreeTreeNode {
    public Integer[] keys;
    public TwoThreeTreeNode[] children;

    // Creates a 2-node
    public TwoThreeTreeNode(Integer key) {
        this.keys = new Integer[]{key, null};
        this.children = new TwoThreeTreeNode[]{null, null, null}; // Ordered from left to right
    }
}
```

Assumptions:

- In a 2-node, `keys[1]` and `children[2]` are `null`.
- In a 3-node, `keys[0]` and `keys[1]` are not `null`, and `keys[0]` $<$ `keys[1]`.

```
public class LLRB {
    public LLRBNode root;

    public class LLRBNode {
        public int key;
        public LLRBNode left, right;
        public boolean isRed;

        public LLRBNode(int key, boolean isRed) {
            this.key = key;
            this.isRed = isRed;
        }
    }

    public LLRB(TwoThreeTreeNode node) {
        root = convert(node);
    }
```

```
    private LLRBNode convert(TwoThreeTreeNode root) {

        if (root == null) {
            return null;
        }

        if (root.keys[1] == null) {

            LLRBNode node = new LLRBNode(root.keys[0], false);

            node.left = convert(root.children[0]);

            node.right = convert(root.children[1]);
            return node;
        }

        LLRBNode leftChild = new LLRBNode(root.keys[0], true);

        LLRBNode parent = new LLRBNode(root.keys[1], false);

        parent.left = leftChild;

        leftChild.left = convert(root.children[0]);

        leftChild.right = convert(root.children[1]);

        parent.right = convert(root.children[2]);

        return parent;
    }
}
```

Alternate answers:

Blank 1 can be any of these, since they all identify a black node:

- `root.keys[1] == null`
- `root.children[2] == null`
- `root.keys[1] == null && root.children[2] == null`
- `root.keys[1] == null || root.children[2] == null`
- `root.keys[0] != null && root.keys[1] == null`
- Other alternate answers might exist.

Blanks 3-4 and Blanks 5-6 can be swapped. Inside the if-statement, you could instead do:

```
node.right = convert(root.children[1])

node.left = convert(root.children[0])
```

Blanks 9-10, 11-12, and 13-14 can be swapped. The order of these three lines don't matter, as long as they all appear:

```
parent.right = convert(root.children[2]);
```

```
leftChild.left = convert(root.children[0]);
```

```
leftChild.right = convert(root.children[1]);
```

Also, `leftChild.left` and `parent.left.left` are equivalent. Similarly, `leftChild.right` and `parent.left.right` are equivalent.

5 Yokeyrin's Cave

(12 points)

Anirudh the Adventurer is on a quest to obtain the 61B Midterm 2 answer key. He ventures into Mastermind Yokeyrin's cave with 100 rooms, each numbered 1 to 100. He starts in Room 1, the cave's entrance, and he must reach Room 100 in as little time as possible to obtain the answer key!

From each room, Anirudh can move to any adjacent room in one minute. Fill out the `shortestTime` method, which is defined as follows:

- Input: `Map<Integer, Set<Integer>> adj` which maps each room (1−100) to a set of all adjacent rooms.
- Output: The minimum number of minutes needed to go from Room 1 to Room 100. If it is impossible to reach Room 100, return −1.

```
public class Queue<T> {

    /** Creates a new queue */
    public Queue() { ... }

    /** Adds item to the back of the queue */
    public void add(T item) { ... }

    /** Removes and returns the front item of the queue */
    public T remove() { ... }

    /** Returns true if the queue has no elements */
    public boolean isEmpty() { ... }
}

public class CaveStory {
    private static class State {
        public int room;
        public int mins;

        public State(int room, int mins) {
            this.room = room;
            this.mins = mins;
        }
    }
```

```
    public static int shortestTime (Map<Integer, Set<Integer>> adj) {
        Set<Integer> traversed = new HashSet<>();
        Queue<State> options = new Queue<>();

        options.add(new State(1, 0));

        while (!options.isEmpty()) {

            State curr = options.remove();

            if (curr.room == 100) {

                return curr.mins;
            }

            if (!traversed.contains(curr.room)) {

                traversed.add(curr.room);

                for (int neighbor : adj.get(curr.room)) {

                    options.add(new State(neighbor, curr.mins + 1);
                }
            }
        }
        return -1; // No path exists
    }
}
```

Alternate answers:

Blanks 1-2 can instead be:

`if (adj.get(curr.room).contains(100))`

`return curr.mins + 1`

Blank 3 can instead be one of these:

- `!traversed.contains(curr.room) && !adj.get(curr.room).isEmpty()`
- `!traversed.contains(curr.room) && adj.containsKey(curr.room)`

Blank 5 can be `Integer neighbor` instead of `int neighbor`. You can also rename `neighbor` to anything you want, as long as it's consistent with Blank 7 (and it doesn't conflict with any other variable names).

Note that `curr` instead of `curr.room` or `curr.mins` was not worth points, because we could not tell which instance variable (`room` or `mins`) you were trying to reference.

Nothing on this page is worth any points.

Bonus Question

(0 Points)

For the following modification to `pairAllStudents` from Question 3, find a list of at most five password hashes that would cause `pairAllStudents` to infinite-loop. If no set of password hashes can cause an infinite loop, write "Impossible":

The StrangeHashSet resizes when the **average number of elements in a bucket is strictly greater than two**. After inserting all items into the StrangeHashSet, if a bucket contains three or more elements, recursively pair students within that bucket using `pairAllStudents`, leaving a single student unpaired if the bucket contained an odd number of students.

```
[0, 2, 4]
```

The goal is to get 3 items all in the same bucket after the first resize. For example, the intended answer results in:

- Bucket 0: `[0, 2, 4]`
- Bucket 1: `[]`

This causes the average number of elements per bucket to be 1.5, because one bucket has 3 items, and the other has 0 items.

At this point, no further resizing occurs (we only resize if the average exceeds 2). We now need to recursively run `pairAllStudents` on the zeroth bucket.

In other words, `pairAllStudents([0, 2, 4])` calls `pairAllStudents([0, 2, 4])`. This causes an infinite loop.

Non-exhaustive list of alternate answers:

- `[1, 3, 5]`

Feedback

(0 Points)

Leave any feedback, comments, concerns, or more drawings below!